Assignment 3: Graph Algorithms for a Campus Festival

Course: Design and Analysis of Algorithms

1. Purpose and scope

This assignment applies graph algorithms to three aspects of campus festival planning: connecting venues, organising preparation tasks and finding delivery routes. Cable connections are undirected; task dependencies and delivery routes are directed. Each part uses a separate input graph.

Implement six algorithms in the supplied GitHub template: DSU, Kruskal, Prim, Kosaraju SCC, DFS topological ordering and shortest paths in a directed acyclic graph (DAG). Choose one language: Java, Python, C++ or C#.

The completed submission must contain the implementations, student tests and an analysis in ANALYSIS.md. The template provides the main program, JSON input/output adapters, examples and public tests.

2. Work plan

1. Create your repository using **Use this template → Create a new repository**. If the repository is private, give the instructor access.

2. Set the language field in submission.json to Java, Python, Cpp or CSharp. Implement the six methods in the corresponding file below.

3. Complete Parts 1–5, run the public tests and add your own test cases.

4. Compare Kruskal and Prim, demonstrate route reconstruction and complete ANALYSIS.md.

5. Submit the commit link and its successful Public checks run in Moodle.

Language	Implementation file
Java	Java/source/Algorithms.java
Python	Python/source/algorithms.py
C++	Cpp/source/algorithms.hpp
C#	CSharp/source/Algorithms.cs

Implement dsu, kruskal, prim, scc, topo and dag. Helper functions, classes and additional source files may be placed directly in the selected source folder. Standard collections, sorting and priority queues are permitted; the required algorithms must be implemented by the student.

3. General requirements

Vertices are numbered from 0 to n-1. An edge ID is its position in the input edges array, starting at 0. The program must preserve the supplied JSON interface, produce one result per request and reset algorithm state between requests. Use standard output for results only.

Support up to 100,000 vertices and 250,000 edges or DSU operations. For weighted graphs, edge weights range from -1,000,000,000 to 1,000,000,000. Use integer arithmetic that safely represents signed 64-bit totals and distances. DFS implementations must handle deep graphs without a call-stack failure. Exact request formats and input limits are specified in CONTRACTS.md.

In the complexity requirements, n denotes the number of vertices, m the number of edges and q the number of DSU operations. Memory bounds include the returned result.

4. Part 1: Connectivity with DSU

Festival venues are connected by cables added one at a time. After each addition, determine the number of connected components and the size of the largest component.

Implement dsu using disjoint-set union with path compression and union by size or rank. Initially, each vertex forms a separate component. Repeated unions and self-unions must leave an existing component unchanged.

Input: n and operations, a sequence of vertex pairs. **Output:** states, with one [components, largest] pair after each operation. For n=0, operations and states are empty.

Example: for n=4 and operations [[0,1],[1,2],[0,2],[3,3]], return states=[[3,2],[2,3],[2,3],[2,3]]. The last two operations do not merge components.

Required complexity: O((n+q) α(n)) amortised time and O(n+q) memory.

5. Part 2: Minimum-cost cable network

Given an undirected weighted graph, find a minimum spanning forest. Within every connected component, the selected edges must connect all vertices without cycles and have minimum total weight. Isolated vertices count as components.

Provide two separate implementations: kruskal, using sorted edges and DSU; and prim, using adjacency lists and a priority queue. Prim must restart at an unvisited vertex until every component has been processed.

Input: n and edges, with each edge represented by [u,v,weight]. **Output:** cost, edge_ids and components. Each selected edge ID must occur once. Parallel edges and negative weights are permitted; self-loops must not be selected. Any optimal forest is accepted. For n=0, return cost=0, edge_ids=[] and components=0.

Example: n=4, edges=[[0,1,4],[1,2,2],[0,2,3]]. Selecting edge IDs [1,2] gives cost=5 and components=2; vertex 3 is isolated. When optimal forests are tied, Kruskal and Prim may return different valid edge IDs.

Required complexity: Kruskal O(n+m log(m+1)); Prim O(n+m log(m+1)) with a lazy heap, or O((n+m) log(n+1)) with an indexed heap. Memory: O(n+m).

6. Part 3: Strongly connected components

In a preparation graph, an edge u→v means that task u must be completed before task v. Identify the strongly connected components (SCCs): maximal groups of vertices that are mutually reachable along directed edges.

Implement scc using Kosaraju's algorithm, with two DFS passes and the transposed graph. Return a component label for every vertex and the condensation graph. Component labels may be any nonnegative integers. Equal labels must identify exactly one SCC.

Input: n and directed edges [u,v]. **Output:** component and condensation. Each condensation edge is a pair of component labels. Include each directed pair between distinct components once; exclude internal edges and repeated pairs. Isolated vertices form separate SCCs. Duplicate edges and self-loops are permitted. For n=0, both arrays are empty.

Example: edges 0→1, 1→0, 1→2, 0→2, 2→3 allow component=[5,5,7,9] and condensation=[[5,7],[7,9]]. The two edges from the first SCC to vertex 2 produce one condensation edge.

Required complexity: O(n+m) expected time when a hash set removes duplicate component pairs; O(n+m) memory.

7. Part 4: Preparation order

For a directed dependency graph, determine whether all preparation tasks can be scheduled. If so, return a topological order in which every prerequisite precedes its dependent task.

Implement topo using DFS with unvisited, active and finished states. An edge to an active vertex indicates a cycle. Reverse the DFS finishing order to obtain the result for an acyclic graph.

Input: n and directed edges [u,v]. **Output:** acyclic and order. If any component contains a cycle, return acyclic=false and order=[]. Otherwise, return acyclic=true and a valid order containing every vertex exactly once, including isolated vertices. For n=0, return acyclic=true and order=[]. A self-loop is a cycle.

Example: edges 0→2, 1→2, 2→3 allow order=[0,1,2,3] or [1,0,2,3].

Required complexity: O(n+m) time and memory.

8. Part 5: Shortest delivery routes

Given a directed weighted graph and a source vertex, find the shortest paths from the source using DAG shortest-path processing. Negative edge weights are permitted.

Implement dag. Check the entire graph for cycles, then process an acyclic graph in topological order and relax outgoing edges from reachable vertices only. A cycle must be rejected even if it is unreachable from the source.

Input: n, edges [u,v,weight] and source, with n≥1 and 0≤source<n. **Output:** acyclic, distance and parent_edge. For cyclic input, return acyclic=false and two empty arrays. For an acyclic graph, return acyclic=true and one distance and parent edge per vertex. The source has distance=0 and parent_edge=-1; an unreachable vertex has distance=null and parent_edge=-1. For every other vertex, parent_edge identifies an incoming edge on a shortest path from the source. Any shortest path is accepted when costs tie.

Example: n=5, source=0, edges=[[0,1,3],[0,2,8],[1,2,-7],[2,3,4]]. Return distance=[0,3,-4,0,null] and parent_edge=[-1,0,2,3,-1]. The route to vertex 3 is 0→1→2→3, with total cost 0.

Add a helper that reconstructs the vertex sequence of a route from parent_edge. Demonstrate its behaviour for a reachable vertex, the source and an unreachable vertex in your tests or analysis. Parallel edges are permitted; a self-loop makes the graph cyclic.

Required complexity: O(n+m) time and memory; O(route length) for route reconstruction.

9. Testing and analysis

After pushing changes, open **Actions → Public checks** in your repository. The supplied workflow compiles or launches the selected implementation and runs the public algorithm tests. The submitted commit must have a successful Public checks run. Optional local commands are documented in TESTING.md.

Add your own cases to tests/student/, recording the input, expected result and actual result. Cover the applicable cases: empty and disconnected graphs, isolated vertices, repeated edges or

operations, self-loops, negative weights, cycles, unreachable vertices and deep DFS traversal.

Complete ANALYSIS.md in English. For each of the six algorithms, provide 3–5 sentences explaining its main steps, state or invariant, relevant edge cases, and time and memory complexity. Explain how state is reset and how deep DFS traversal is handled.

Compare Kruskal and Prim on the same graphs at three input sizes. Record n and m, reproducible input generation, commands or Actions links, at least three timing measurements per algorithm at each size, and median times. Verify equal minimum costs and valid forests, then explain the observed performance. Include the route reconstruction examples and acknowledge sources and assistance according to the syllabus.

After the deadline, the instructor will run additional tests on the submitted commit and review the required implementations and analysis.

10. Submission

Submit two links in Moodle's **Online text** field:

1. The GitHub commit containing the completed implementations, analysis and student tests. Open that commit and copy its URL.

2. The successful **Public checks** run for the same commit. Open the run in Actions and copy its URL.

Complete submission using **Submit assignment**. The commit linked in Moodle is the version assessed. Before the deadline, any replacement submission must include both updated links. A private repository must remain accessible to the instructor.